ACTIVITY SHEET – 9

Programs Using try, catch, finally

1. OBJECTIVES

By completing this activity, students will be able to:

• Understand exception handling in Java
• Use try, catch, and finally blocks effectively
• Handle runtime errors gracefully
• Design robust and fault-tolerant programs
• Apply exception handling in real-world scenarios

2. READING REQUIREMENTS

Oracle Java Tutorials – Exceptions:
https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html

3. ILLUSTRATIVE EXAMPLES

Example 1: Basic try-catch

Code:

```
public class Test {
   public static void main(String[] args) {
      try {
         int a = 10 / 0;
      } catch (ArithmeticException e) {
         System.out.println("Cannot divide by zero");
      }
   }
}
```

Example 2: try-catch-finally

Code:

```
public class TestFinally {
   public static void main(String[] args) {
      try {
         int arr[] = new int[3];
         arr[5] = 10;
```

```
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Index error");
        } finally {
            System.out.println("Execution completed");
        }
    }
}
```

4. PROGRAMMING / TASK QUESTIONS

A. EASY LEVEL (Conceptual)

1. Write a Java program to handle division by zero using try-catch.

Hint: Use Arithmetic Exception.

JAVA CODE:

Welcome | DivisionHandler.java

DivisionHandler.java > ...

```java
import java.util.Scanner;

public class DivisionHandler {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println(x: "Enter two integers:");
        int a = sc.nextInt();
        int b = sc.nextInt();

        try {
            int result = a / b;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println(x: "Cannot divide by zero");
        }

        sc.close();
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac DivisionHandler.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java DivisionHandler.java
Enter two integers:
10
2
5

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac DivisionHandler.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java DivisionHandler.java
Enter two integers:
10
0
Cannot divide by zero
```

2.Write a program to handle array index out of bounds exception.
Hint: Access invalid index.

JAVA CODE:

Welcome | DivisionHandler.java | ArrayIndexHandler.java

ArrayIndexHandler.java > ...

```java
import java.util.Scanner;

public class ArrayIndexHandler {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println(x: "Enter size of array:");
        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println(x: "Enter elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println(x: "Enter index to access:");
        int index = sc.nextInt();

        try {
            System.out.println(arr[index]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(x: "Invalid array index");
        }

        sc.close();
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac ArrayIndexHandler.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java ArrayIndexHandler.java
Enter size of array:
5
Enter elements:
10 20 30 40 50
Enter index to access:
2
30

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac ArrayIndexHandler.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java ArrayIndexHandler.java
Enter size of array:
5
Enter elements:
1 2 3 4 5
Enter index to access:
6
Invalid array index
```

3. Write a program to demonstrate finally block execution.
Hint: finally executes always.

JAVA CODE:

Welcome | FinallyDemo.java

FinallyDemo.java > ...

```java
import java.util.Scanner;

public class FinallyDemo {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println(x: "Enter two integers:");
        int a = sc.nextInt();
        int b = sc.nextInt();

        try {
            int result = a / b;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println(x: "Cannot divide by zero");
        } finally {
            System.out.println(x: "Finally block executed");
        }

        sc.close();
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac FinallyDemo.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java FinallyDemo.java
Enter two integers:
102
100
1
Finally block executed

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac FinallyDemo.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java FinallyDemo.java
Enter two integers:
10
0
Cannot divide by zero
Finally block executed
```

B. MEDIUM LEVEL (Problem-Based)

4. Design a program where user inputs two numbers and performs division. Handle invalid inputs and division errors.
Hint: Use try-catch for multiple exceptions.

JAVA CODE:

Welcome | FinallyDemo.java | MultipleExceptionHandler.java

MultipleExceptionHandler.java > ...

```java
import java.util.Scanner;

public class MultipleExceptionHandler {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println(x: "Enter two numbers:");
        try {
            int a = sc.nextInt();
            int b = sc.nextInt();

            int result = a / b;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println(x: "Cannot divide by zero");
        } catch (Exception e) {
            System.out.println(x: "Invalid input");
        }

        sc.close();
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac MultipleExceptionHandler.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java MultipleExceptionHandler.java
Enter two numbers:
10
2
5

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac MultipleExceptionHandler.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java MultipleExceptionHandler.java
Enter two numbers:
10
0
Cannot divide by zero

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac MultipleExceptionHandler.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java MultipleExceptionHandler.java
Enter two numbers:
10
abc
Invalid input
```

5. A file reading system may encounter errors like file not found or invalid format. Design a program to handle such exceptions gracefully.
Hint: Use multiple catch blocks.

JAVA CODE:

LoginSystem.java ● | ResourceManagementDemo.java ● | NumberProcessor.java | OnlineTransactionSystem.java

FileExceptionHandling.java > ...

```java
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
class FileExceptionHandling {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print(s: "Enter file name: ");
            String fileName = sc.nextLine();
            File file = new File(fileName);
            Scanner fileReader = new Scanner(file);
            String data = fileReader.nextLine();
            int number = Integer.parseInt(data);
            System.out.println("Number read from file = " + number);
            fileReader.close();
        }
        catch (FileNotFoundException e) {
            System.out.println(x: "ERROR: File not found!");
        }
        catch (NumberFormatException e) {
            System.out.println(x: "ERROR: Invalid file format! File must contain a valid integer.");
        }
        catch (Exception e) {
            System.out.println(x: "ERROR: Something went wrong while reading the file.");
        }
        finally {
            System.out.println(x: "FILE HANDLING COMPLETED.");
            sc.close();
        }
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac FileExceptionHandling.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java FileExceptionHandling.java
Enter file name: data.txt
ERROR: File not found!
FILE HANDLING COMPLETED.
```

6. Create a banking system where withdrawal may fail due to insufficient balance. Handle this using exception handling.
Hint: Throw and catch exception.

JAVA CODE:

Welcome | FinallyDemo.java | MultipleExceptionHandler.java | FileReadHandler.java | Banking

BankingSystem.java > ...

```java
import java.util.Scanner;
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String msg) {
        super(msg);
    }
}
public class BankingSystem {
    static void withdraw(int balance, int amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException(msg: "Insufficient balance");
        } else {
            balance = balance - amount;
            System.out.println(x: "Withdrawal successful");
            System.out.println("Remaining balance: " + balance);
        }
    }
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println(x: "Enter balance:");
        int balance = sc.nextInt();
        System.out.println(x: "Enter withdrawal amount:");
        int amount = sc.nextInt();
        try {
            withdraw(balance, amount);
        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }
        sc.close();
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac BankingSystem.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java BankingSystem.java
Enter balance:
300000
Enter withdrawal amount:
30000
Withdrawal successful
Remaining balance: 270000

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac BankingSystem.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java BankingSystem.java
Enter balance:
2000
Enter withdrawal amount:
200000
Insufficient balance
```

C. HARD LEVEL (Design / Placement-Oriented)

7. Design a login system that validates user credentials. Handle cases such as null input, invalid format, and authentication failure using exceptions.

Hint: Use custom exception handling.

JAVA CODE:

Welcome | LoginSystem.java

LoginSystem.java > LoginSystem > main(String[])

```java
import java.util.Scanner;

class InvalidFormatException extends Exception {
    InvalidFormatException(String msg) {
        super(msg);
    }
}

class AuthenticationException extends Exception {
    AuthenticationException(String msg) {
        super(msg);
    }
}

public class LoginSystem {

    static void validate(String username, String password) throws InvalidFormatException, AuthenticationException {
        if (username == null || password == null || username.isEmpty() || password.isEmpty()) {
            throw new NullPointerException(s: "Input cannot be null or empty");
        }

        if (!username.matches(regex: "[a-zA-Z0-9]+")) {
            throw new InvalidFormatException(msg: "Invalid username format");
        }

        String correctUser = "admin";
        String correctPass = "1234";

        if (!username.equals(correctUser) || !password.equals(correctPass)) {
            throw new AuthenticationException(msg: "Authentication failed");
        }

        System.out.println(x: "Login successful");
    }
```

Welcome | LoginSystem.java

LoginSystem.java > ...

```java
public class LoginSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println(x: "Enter username:");
        String username = sc.nextLine();

        System.out.println(x: "Enter password:");
        String password = sc.nextLine();

        try {
            validate(username, password);
        } catch (NullPointerException e) {
            System.out.println(e.getMessage());
        } catch (InvalidFormatException e) {
            System.out.println(e.getMessage());
        } catch (AuthenticationException e) {
            System.out.println(e.getMessage());
        }

        sc.close();
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac LoginSystem.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java LoginSystem.java
Enter username:
admin
Enter password:
1234
Login successful

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac LoginSystem.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java LoginSystem.java
Enter username:
pavani
Enter password:
1234
Authentication failed

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java LoginSystem.java
Enter username:

Enter password:

Input cannot be null or empty

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java LoginSystem.java
Enter username:
12cth0
Enter password:
1234
Authentication failed

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java LoginSystem.java
Enter username:
@^2o
Enter password:
5fg
Invalid username format
```

8. Implement a system where multiple operations are performed and ensure resources are always released using finally block.
Hint: finally for cleanup.

JAVA CODE:

```
Welcome    LoginSystem.java    ResourceManagementDemo.java
ResourceManagementDemo.java > ...
import java.util.Scanner;
public class ResourceManagementDemo {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println(x: "Enter two integers:");
        int a = sc.nextInt();
        int b = sc.nextInt();
        try {
            int result = a / b;
            System.out.println("Result: " + result);
            int[] arr = {10, 20, 30};
            System.out.println("Access element: " + arr[5]);
        } catch (ArithmeticException e) {
            System.out.println(x: "Division by zero error");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(x: "Array index error");
        } catch (Exception e) {
            System.out.println(x: "General error");
        } finally {
            System.out.println(x: "Releasing resources");
            sc.close();
        }
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac ResourceManagementDemo.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java ResourceManagementDemo.java
Enter two integers:
10
0
Division by zero error
Releasing resources

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac ResourceManagementDemo.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java ResourceManagementDemo.java
Enter two integers:
10
2
Result: 5
Array index error
Releasing resources
```

9. Design a program to process a list of numbers where some inputs may be invalid. Continue processing remaining elements even after encountering exceptions.
Hint: Use try-catch inside loop.

JAVA CODE:

Welcome | LoginSystem.java | ResourceManagementDemo.java | Numbe

NumberProcessor.java > ...

```java
import java.util.Scanner;
public class NumberProcessor {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println(x: "Enter number of inputs:");
        int n = sc.nextInt();
        String[] inputs = new String[n];
        System.out.println(x: "Enter values:");
        for (int i = 0; i < n; i++) {
            inputs[i] = sc.next();
        }
        for (int i = 0; i < n; i++) {
            try {
                int num = Integer.parseInt(inputs[i]);
                System.out.println("Valid number: " + num);
            } catch (Exception e) {
                System.out.println("Invalid input: " + inputs[i]);
            }
        }
        sc.close();
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac NumberProcessor.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java NumberProcessor.java
Enter number of inputs:
4
Enter values:
10
20
30
40
Valid number: 10
Valid number: 20
Valid number: 30
Valid number: 40

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac NumberProcessor.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java NumberProcessor.java
Enter number of inputs:
3
Enter values:
10
!
q
Valid number: 10
Invalid input: !
Invalid input: q
```

10. Create a program simulating an online transaction system where multiple exceptions can occur. Ensure system stability using structured exception handling.
Hint: Combine try-catch-finally.

JAVA CODE:

Welcome | LoginSystem.java | ResourceManagementDemo.java | NumberProcessor.java | OnlineTransact

OnlineTransactionSystem.java > ...

```java
import java.util.Scanner;

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String msg) {
        super(msg);
    }
}

public class OnlineTransactionSystem {

    static void processTransaction(int balance, int amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException(msg: "Insufficient balance");
        }
        int result = balance - amount;
        System.out.println(x: "Transaction successful");
        System.out.println("Remaining balance: " + result);
    }

    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println(x: "Enter balance:");
        try {
            int balance = sc.nextInt();

            System.out.println(x: "Enter transaction amount:");
            int amount = sc.nextInt();

            processTransaction(balance, amount);
```

```
Welcome    LoginSystem.java    ResourceManagementDemo.java    NumberProc
OnlineTransactionSystem.java > ...
public class OnlineTransactionSystem {
    public static void main(String[] args) {
        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        } catch (ArithmeticException e) {
            System.out.println(x: "Arithmetic error occurred");
        } catch (Exception e) {
            System.out.println(x: "Invalid input");
        } finally {
            System.out.println(x: "Transaction process completed");
            sc.close();
        }
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>javac OnlineTransactionSystem.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java OnlineTransactionSystem.java
Enter balance:
12345
Enter transaction amount:
1234
Transaction successful
Remaining balance: 11111
Transaction process completed

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java OnlineTransactionSystem.java
Enter balance:
123
Enter transaction amount:
1234
Insufficient balance
Transaction process completed

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java OnlineTransactionSystem.java
Enter balance:
asdf
Invalid input
Transaction process completed

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_9>java OnlineTransactionSystem.java
Enter balance:
-89
Enter transaction amount:
57
Insufficient balance
Transaction process completed
```

5. ASSIGNMENT EVALUATION

0 : Not Done
1 : Incomplete
2 : Late Completion
3 : Needs Improvement
4 : Complete
5 : Well Done

6. VIVA QUESTIONS

1. What is an exception?

2. What is try-catch block?

3. What is finally block?

4. Difference between checked and unchecked exceptions?

5. Can finally block execute without catch?

6. What happens if exception is not handled?

7. What is custom exception?